

Unigram Tokenization

Unigram algorithm SentencePiece এর সাথে ব্যবহার করা হয়, যা ALBERT, T5, mBART, Big Bird এবং XLNet এর মতো মডেলগুলোতে ব্যবহৃত হয়। SentencePiece এই সমস্যাটা সমাধান করে যে সব ভাষায় শব্দ আলাদা করতে স্পেস ব্যবহার হয় না। এটি পুরো ইনপুটকে raw input stream হিসেবে নেয়, যেখানে স্পেসকেও একটি character হিসেবে ধরা হয়। এরপর Unigram algorithm ব্যবহার করে একটি উপযুক্ত vocabulary তৈরি করে। এই অংশে Unigram খুব গভীরভাবে বোঝানো হয়েছে, এমনকি সম্পূর্ণ implementation পর্যন্ত দেখানো হয়েছে। যদি শুধু overview দরকার হয় তাহলে শেষে চলে যাওয়া যায়।

Training Algorithm

BPE এবং WordPiece এর তুলনায় Unigram উল্টোভাবে কাজ করে। এটি শুরুতে একটি বড় vocabulary নেয় এবং ধীরে ধীরে token বাদ দিতে থাকে যতক্ষণ না desired vocabulary size পাওয়া যায়। এই initial vocabulary তৈরি করার জন্য কয়েকটি উপায় আছে, যেমন pre-tokenized word থেকে সবচেয়ে common substring নেওয়া বা বড় vocabulary size দিয়ে BPE apply করা। প্রতিটি training step এ Unigram algorithm পুরো corpus এর উপর একটি loss হিসাব করে। তারপর vocabulary এর প্রতিটি symbol এর জন্য হিসাব করে যে সেটি বাদ দিলে loss কত বাড়বে। যেসব symbol বাদ দিলে loss সবচেয়ে কম বাড়ে, সেগুলোকে কম গুরুত্বপূর্ণ ধরা হয় এবং আগে বাদ দেওয়া হয়। এই পুরো process অনেক costly, তাই একবারে একটি symbol বাদ না দিয়ে p শতাংশ symbol (যেখানে p একটি hyperparameter, সাধারণত 10 বা 20) একসাথে বাদ দেওয়া হয়। এই process বারবার করা হয় যতক্ষণ না desired vocabulary size পাওয়া যায়। তবে base character কখনোই বাদ দেওয়া হয় না যাতে যেকোনো শব্দ tokenize করা যায়।

এখনও বিষয়টা একটু অস্পষ্ট লাগতে পারে, কারণ মূল কাজ হলো loss হিসাব করা এবং token বাদ দিলে সেটা কিভাবে বদলায় তা দেখা, কিন্তু এটা কিভাবে করা হয় তা এখনো বোঝানো হয়নি। এই অংশটা নির্ভর করে Unigram tokenization algorithm এর উপর, তাই এখন আমরা সেটা দেখব।

আমরা আগের উদাহরণের corpus আবার ব্যবহার করি: ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5) এবং initial vocabulary হিসেবে সব substring নেওয়া হয়: ["h", "u", "g", "hu", "ug", "p", "pu", "n", "un", "b", "bu", "s", "hug", "gs", "ugs"]

Tokenization Algorithm

Unigram model একটি language model যেখানে প্রতিটি token স্বাধীন, অর্থাৎ আগের token এর উপর নির্ভর করে না। কোনো token X এর probability শুধু X এর নিজের probability। তাই যদি এই model দিয়ে text generate করা হয়, তাহলে সবসময় সবচেয়ে common token বেশি আসবে।

কোনো token এর probability = তার frequency / সব token এর মোট frequency। যেমন "ug" token টি "hug", "pug", এবং "hugs" এ আছে, তাই এর frequency 20। সব subword এর frequency হলো: ("h", 15) ("u", 36) ("g", 20) ("hu", 15) ("ug", 20) ("p", 17) ("pu", 17) ("n", 16) ("un", 16) ("b", 4) ("bu", 4) ("s", 5) ("hug", 15) ("gs", 5) ("ugs", 5) মোট frequency = 210, তাই "ug" এর probability = 20/210।

এখন একটি word tokenize করতে হলে সব possible segmentation বের করা হয় এবং প্রতিটির probability হিসাব করা হয়। যেহেতু token গুলো independent, তাই total probability = সব token এর probability এর গুণফল। উদাহরণ: "pug" $\rightarrow$ ["p","u","g"] হলে probability = $5/210 \times 36/210 \times 20/210 = 0.000389$ । আবার ["pu","g"] হলে probability = $5/210 \times 20/210 = 0.0022676$, যা বেশি। তাই কম token ব্যবহার করা segmentation সাধারণত বেশি probability পায়।

"pug" এর possible segmentation: ["p","u","g"] : 0.000389, ["p","ug"] : 0.0022676, ["pu","g"] : 0.0022676। তাই final tokenization হবে ["p","ug"] বা ["pu","g"] (যেটা আগে পাওয়া যায়)।

Viterbi Algorithm

সব possible segmentation বের করা সবসময় সহজ না, তাই Viterbi algorithm ব্যবহার করা হয়। এখানে একটি graph তৈরি করা হয় যেখানে character a থেকে b পর্যন্ত substring vocabulary তে থাকলে একটি branch থাকে এবং সেই branch এ probability assign করা হয়। Viterbi algorithm প্রতিটি position এ best score এর segmentation বের করে। শুরু থেকে শেষ পর্যন্ত গিয়ে শেষে best path বের করে final tokenization নির্ধারণ করা হয়।

উদাহরণ: "unhug" এর জন্য best segmentation: Character 0: "u" (0.171429), Character 1: "un" (0.076191), Character 2: "un h" (0.005442), Character 3: "un hu" (0.005442), Character 4: "un hug" (0.005442)। তাই tokenization = ["un","hug"]।

Back to Training (Loss Calculation)

Training এর সময় প্রতিটি word tokenize করে তার score বের করা হয়। তারপর loss হিসাব করা হয় negative log likelihood দিয়ে, অর্থাৎ সব word এর $-\log(P(\text{word}))$ এর যোগফল। উদাহরণ: "hug": ["hug"] (0.071428), "pug": ["pu","g"] (0.007710), "pun": ["pu","n"] (0.006168), "bun": ["bu","n"] (0.001451), "hugs": ["hug","s"] (0.001701)।

$$\text{Loss} = 10 \times (-\log(0.071428)) + 5 \times (-\log(0.007710)) + 12 \times (-\log(0.006168)) + 4 \times (-\log(0.001451)) + 5 \times (-\log(0.001701)) = 169.8$$

Token Removal Effect

এখন দেখা হয় কোন token বাদ দিলে loss কিভাবে পরিবর্তন হয়। যেমন "pu" বাদ দিলে loss একই থাকবে, কারণ "pug" অন্যভাবে tokenize করা যায় একই score দিয়ে। কিন্তু "hug" বাদ দিলে "hug" $\rightarrow$ ["hu","g"] এবং "hugs" $\rightarrow$ ["hu","gs"] হয়ে যায়, ফলে loss বেড়ে যায় 23.5। তাই "pu" বাদ দেওয়া হবে, কিন্তু "hug" রাখা হবে।